基于泰勒多项式展开的神经网络在脑电信号分类中的应用与实现

汇报人：金帅

目录

CONTENTS

01 . Target

02. Backgroud

03. Motivtion

04. Change

05. Methode Design

06. Experimental Results

01

Target

利用“**泰勒多项式特征展开**”**的神经网络**，实现**脑电信号的分类**（**如**癫痫脑电与正常脑电的区分、脑机接口指令识别等）

02

Backgroud

Backgroud

脑电信号特点

即脑电信号蕴含着大脑活动的重要信息，**在**癫痫检测、**精神疾病**诊断、**人机交互等**领域有关键应用价值，**但**这类信号**具有高维度、强非线性、易受噪声干扰的特点**，传统方**法**（**如**单纯的神经网络、**普通多**项式回归）难以高效提取**特征并**实现精准分类

应用价值

脑电信号蕴含大脑活动规律，**在医**疗诊断（癫痫、**抑郁等疾病**检测）、**人机交互**（脑机接口）**等**领域至关重要，**精准分**类能直接推动这些场景的落地

传统方法的局限

1.单纯神经网络：依赖“**深**层堆叠”**提取非**线性特**征**，**但浅**层网络表达能力不足，深层网络又面临**计算成本高**、**梯度消失 / 爆炸**（训练难收敛）的问题。

2.**普通多**项式回归：能捕捉“**高**阶非线性”，**但缺乏**神经网络“**端到端学**习”**的能力**，**泛化性差**（换数据集后效果骤降）。

03

Motivtion

现有方法的痛 点

1

神经网络

若用浅层网络，仅靠Activate**函数的非**线性无法捕捉脑电的高阶规律；若堆深层数，手动实现反向传播时，Activate_grad**返回的梯度易多**层叠加后趋近于 0，导致梯度消失。

2

多项式回归

仅能做静态特征映射，**无法像神**经网络一样通过参数更新适**配数据分布**。

本方案的创新切入点

1.用Taylor_expan函数提前挖掘高阶特征，给网络喂现成规律；

2.用定制化初始化、激活函数、优化器适配增强后的特征，确保网络高效学习。

04

Challenges

1

特征层面

非线性关系捕捉难

大脑是高度复杂的非线性动力学系统，很多过程无法通过简单的线性叠加原理描述，难以捕捉非线性信号。

2

模型层面

神经网络组件协同难

参数初始化、激活函数、优化器相互影响，需协同优化才能避免训练问题。

3

评估层面

小样本下泛化能力保障难

生物信号数据集往往样本量有限，如何通过实验设计（如交叉验证）避免模型“过拟合”，是关键挑战。

05

Method Design

01

泰勒多项式展开 —— 显式转化高阶特征（解决“非线性捕捉难”）

运用函数Taylor_expan：将原始特征空间中的非线性关系转化为高维特征空间中的线性关系。

核心逻辑：泰勒展开的数学思想：任何光滑的非线性函数都可以用多项式（幂函数的组合）近似表示。

当 power=2 时

在 1 阶基础上，增加所有 2 阶交叉特征（即特征间的乘积）：

例如输入特征为 [x1, x2, x3]（3 个特征），2 阶特征包括 x1*x1, x1*x2, x1*x3, x2*x2, x2*x3, x3*x3

最终 X = [常数项行; 原特征x; 2阶交叉特征x2]

```python
def Taylor_expan(x, power):
    if power == 1:
        X = np.vstack([np.ones((1, x.shape[1])), x])
    elif power == 2:
        x2 = []
        xt1 = []
        xt2 = x
        n = x.shape[0]
        for i in range(n):
            xt1 = xt2 * np.repeat(xt2[0:1, :], xt2.shape[0], axis=0)
            xt2 = xt2[1:, :]
            x2.append(xt1)
        x2 = np.vstack(x2)
        X = np.vstack([np.ones((1, x.shape[1])), x, x2])
    elif power == 3:
        x2 = []
        xt1 = []
        xt2 = x
        n = x.shape[0]
```

Gradient_renewal

```
"D:\python caogao\lunwen 1\.venv\Scripts\python.exe" "E:\论文\ResMTN for EEG - 副本\Gradient_renewal.py"

进程已结束，退出代码为 0
```

01

泰勒多项式展开 —— 显式转化高阶特征（解决“非线性捕捉难”）

当 power=3 时
在 2 阶基础上，再增加所有 3 阶交叉特征（即 2 阶特征与原特征的乘积）：
例如在 2 阶特征基础上，与原特征交叉得到 x1*x1*x1, x1*x1*x2, ... 等 3 阶项
最终 X = [常数项行; 原特征x; 2阶特征x2; 3阶特征x3]

最终实现：

1.能将非线性关系转化为更高维度空间中的线性关系
2.保留特征间的交互信息（如 x1*x2 可以表示两个特征的协同作用）
3.阶数越高，能捕捉的非线性复杂度越高

```python
def Taylor_expan(x, power):
    if power == 1:
        X = np.vstack([np.ones((1, x.shape[1])), x])
    elif power == 2:
        x2 = []
        xt1 = []
        xt2 = x
        n = x.shape[0]
        for i in range(n):
            xt1 = xt2 * np.repeat(xt2[0:1, :], xt2.shape[0], axis=0)
            xt2 = xt2[1:, :]
            x2.append(xt1)
        x2 = np.vstack(x2)
        X = np.vstack([np.ones((1, x.shape[1])), x, x2])
    elif power == 3:
        x2 = []
        xt1 = []
        xt2 = x
        n = x.shape[0]
```

Gradient_renewal

```
"D:\python caogao\lunwen 1\.venv\Scripts\python.exe" "E:\论文\ResMTN for EEG - 副本\Gradient_renewal.py"

进程已结束，退出代码为 0
```

02

定制化神经网络组件 —— 适配增强后的特征（解决 组件协同难）

组件一：Initialization函数（权重初始化）

根据输入的参数type选择不同的初始化方式，生成一个形状为(row_num, col_num)的权重矩阵W，以适配网络不同层的需求，提升训练稳定性与效果。

原因：

若权重全部初始化为 0，会导致所有神经元输出相同，反向传播时参数更新一致，失去网络的表达能力。

```python
def Initialization(row_num, col_num, type):
    if type == 1:  # 随机初始化
        W = np.random.rand(row_num, col_num)
    elif type == 2:  # Gaussian初始化
        W = np.sqrt(2 /
                    (row_num + col_num)) * np.random.randn(row_num, col_num)
    elif type == 3:  # Xavier-logistic初始化
        r = np.sqrt(6 / (row_num + col_num))
        W = np.random.uniform(-r, r, (row_num, col_num))
    elif type == 4:  # Xavier-tanh初始化
        r = 4 * np.sqrt(6 / (row_num + col_num))
        W = np.random.uniform(-r, r, (row_num, col_num))
    elif type == 5:  # 随机初始化(0~1)
        W = np.random.rand(row_num, col_num)
    return W
```

lunwen 66

```
"D:\python caogao\lunwen 1\.venv\Scripts\python.exe" "D:\python caogao\lunwen 1\lunwen 66.py"

进程已结束，退出代码为 0
```

02

定制化神经网络组件 —— 适配增强后的特征（解决 组件协同难）

不同初始化方法的适配性：

1.高斯分布和 Xavier 系列初始化通过调整权重的取值范围（基于输入输出维度row_num和col_num），避免权重过大或过小。

2.Xavier 初始化针对不同激活函数（sigmoid/tanh）调整了区间范围，使前向传播的输出和反向传播的梯度保持在合理范围，加速模型收敛。

```python
def Initialization(row_num, col_num, type):
    if type == 1:  # 随机初始化
        W = np.random.rand(row_num, col_num)
    elif type == 2:  # Gaussian初始化
        W = np.sqrt(2 /
                    (row_num + col_num)) * np.random.randn(row_num, col_num)
    elif type == 3:  # Xavier-logistic初始化
        r = np.sqrt(6 / (row_num + col_num))
        W = np.random.uniform(-r, r, (row_num, col_num))
    elif type == 4:  # Xavier-tanh初始化
        r = 4 * np.sqrt(6 / (row_num + col_num))
        W = np.random.uniform(-r, r, (row_num, col_num))
    elif type == 5:  # 随机初始化(0~1)
        W = np.random.rand(row_num, col_num)
    return W
```

lunwen 66

```
"D:\python caogao\lunwen 1\.venv\Scripts\python.exe" "D:\python caogao\lunwen 1\lunwen 66.py"

进程已结束，退出代码为 0
```

02

定制化神经网络组件 —— **适配增**强后的特征（**解决** 组件协同难）

组件二：运用Activate**函数**， 实现神经网络中常用的激**活函数功能**：

对输入数据x进行非线性变换（通过不同的激活函**数**）， **从而**让神经网络能够学习和拟合复杂的非线性关系。

当type == 1时，使用ReLU（Rectified Linear Unit） **激活函数**：X = np.maximum(0, x)， **即**输入大于 0 时输出自身，小于等于 0 时输出 0， **能有效**缓解梯度消失问题，计算效率高。

```python
def Activate(x, type):
    if type == 1:  # RELU
        X = np.maximum(0, x)
    elif type == 2:  # LeakyRELU
        X = np.maximum(0.01 * x, x)
    elif type == 3:  # Logistic
        X = 1 / (1 + np.exp(-x))
    elif type == 4:  # tanh
        X = np.tanh(x)
    return X, type
```

lunwen 66

```
"D:\python caogao\lunwen 1\.venv\Scripts\python.exe" "D:\python caogao\lunwen 1\lunwen 66.py"

进程已结束，退出代码为 0
```

02

定制化神经网络组件 —— 适配增强后的特征（解决 组件协同难）

当type == 2时，使用Leaky ReLU激活函数：X = np.maximum(0.01 * x, x)， 是 ReLU 的改进版，对负输入保留一个微小的梯度（0.01 倍），避免了 ReLU 可能出现的“神经元死亡”问题。

当type == 3时，使用Logistic（Sigmoid） 激活函数：X = 1 / (1 + np.exp(-x))， 将输入映射到 (0,1) 区间，可用于二分类问题的输出层，表示概率。

当type == 4时，使用tanh激活函数：X = np.tanh(x)，将输入映射到 (-1,1) 区间，输出均值接近 0， 在某些场景下收敛速度比 sigmoid 更快。

```python
def Activate(x, type):
    if type == 1:  # RELU
        X = np.maximum(0, x)
    elif type == 2:  # LeakyRELU
        X = np.maximum(0.01 * x, x)
    elif type == 3:  # Logistic
        X = 1 / (1 + np.exp(-x))
    elif type == 4:  # tanh
        X = np.tanh(x)
    return X, type
```

lunwen 66

```
"D:\python caogao\lunwen 1\.venv\Scripts\python.exe" "D:\python caogao\lunwen 1\lunwen 66.py"

进程已结束，退出代码为 0
```

02

定制化神经网络组件 —— 适配增强后的特征（解决 组件协同难）

组件三：利用Gradient_renewal函数，通过梯度下降法更新参数以最小化损失函数。

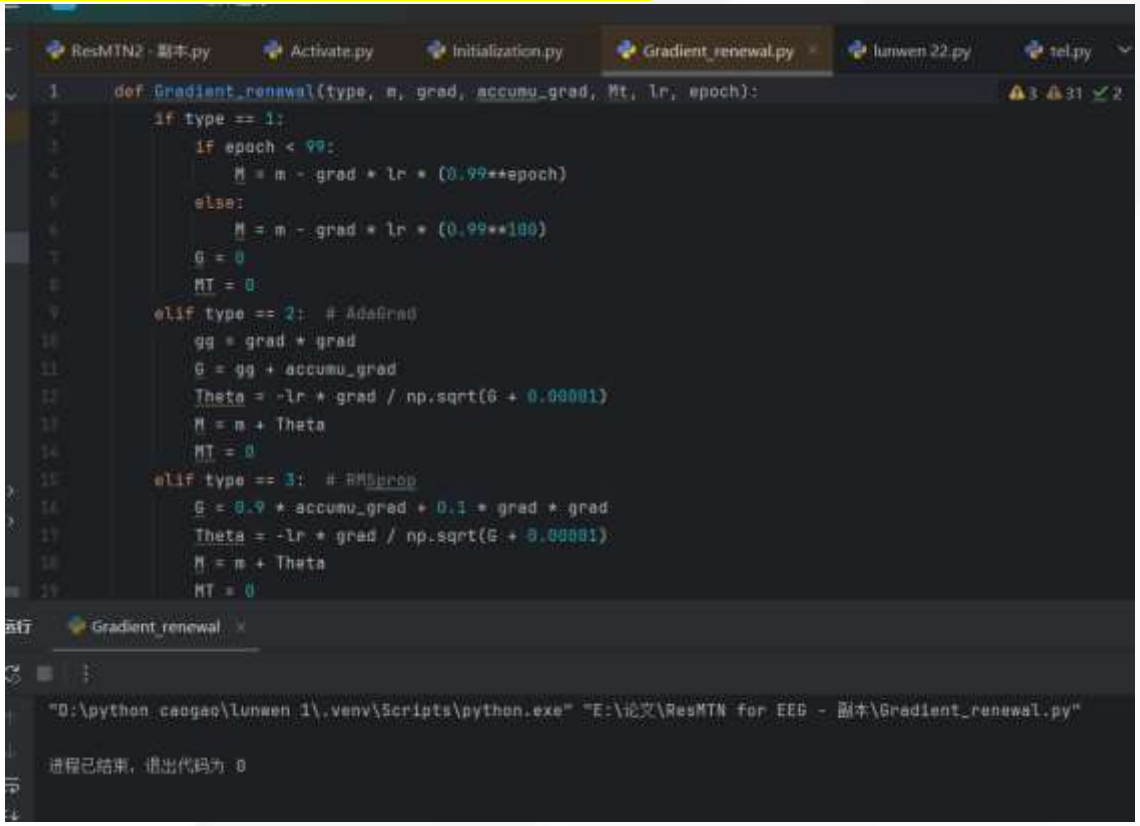
核心：能通过指定type选择不同算法，计算新的参数值

例如：

type == 1：衰减学习率的梯度下降

逻辑：学习率随训练轮次epoch按0.99^epoch衰减（epoch≥99 后固定为0.99^100），参数更新为m - grad * 衰减后的学习率。

作用:通过逐渐减小学习率，避免训练后期参数在最优值附近震荡，提高收敛稳定性。



02

定制化神经网络组件 —— 适配增强后的特征（解决 组件协同难）

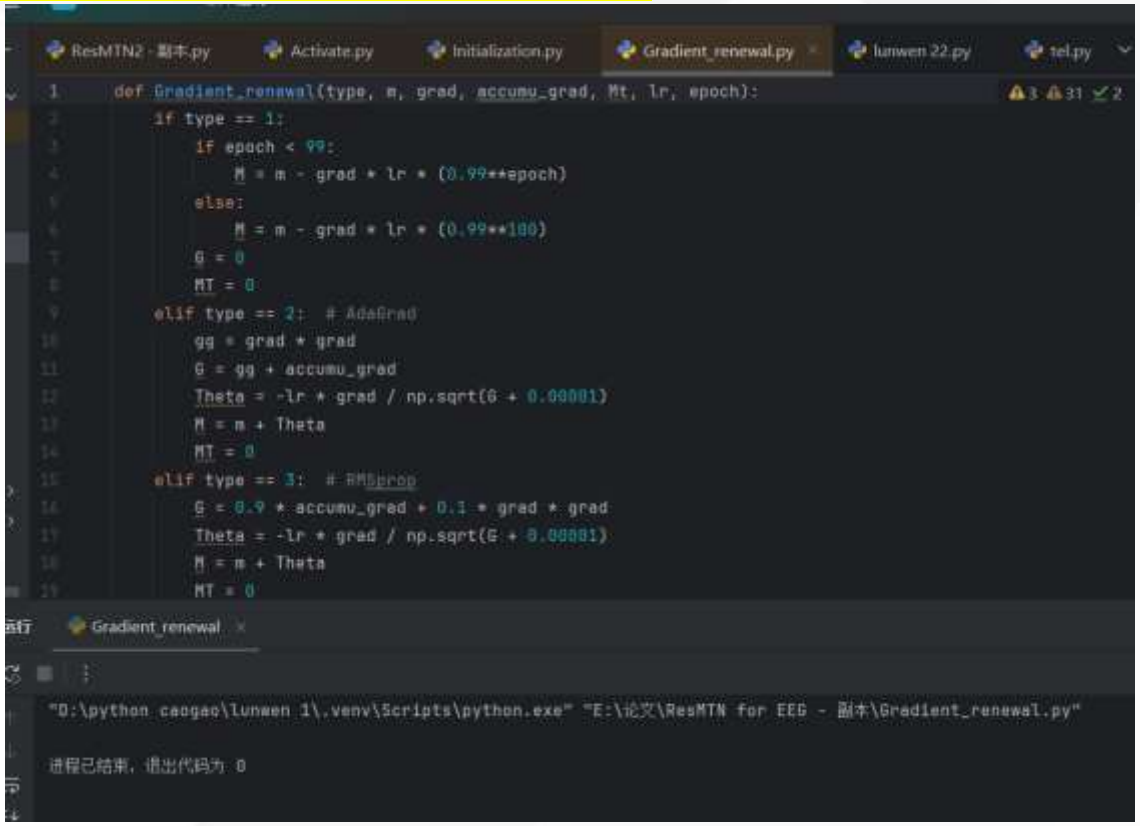
组件三：利用Gradient_renewal函数，通过梯度下降法更新参数以最小化损失函数。（参数优化器）

最终实现：

1.能根据梯度大小动态调整步长，避免手动调参的麻烦。

2.解决初期累积值偏小的问题，提高更新准确性。

3.灵活切换优化策略，适配不同的模型和数据场景，从而更高效地找到损失函数的最优解。



03

第三步：训练与评估策略 —— 保障泛化性（解决“小样本过拟合”）

一.数据预处理（归一化）

核心逻辑：通过公式 $x = \frac{res_Train_Data - T_min}{T_max - T_min}$，**将**脑电数据缩放到统一范围，**消除不同**脑电通道的幅值差异，让特征分布更均匀，避**免因特征分布不均影响模型学**习。

通过 MinMaxScaler 归一化后，所有特征被缩放到 [0, 1] **区**间，每个特征对模型的 **“影响力” 更均衡，模型能公平学**习所有特征的规律，避免因 **“特征幅**值偶然差异” **学**错规律。

```python
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# 假设 res_Train_Data 是加载好的训练数据（形状为 [样本数, 特征数]）
# 这里用随机数模拟脑电数据，实际需替换为真实数据加载逻辑
res_Train_Data = np.random.rand(100, 20)  # 100个样本，20个特征

# 初始化 MinMaxScaler，将数据归一化到 [0, 1] 区间
scaler = MinMaxScaler()
# 对训练数据进行归一化
res_Train_Data_normalized = scaler.fit_transform(res_Train_Data)

# 若有测试数据（res_Test_Data），也需用训练集的 scaler 进行归一化（避免数据泄漏）
res_Test_Data = np.random.rand(30, 20)  # 模拟测试数据
res_Test_Data_normalized = scaler.transform(res_Test_Data)

print("归一化后训练数据示例：\n", res_Train_Data_normalized[:5])
print("归一化后测试数据示例：\n", res_Test_Data_normalized[:5])
```

```
0.64457957 0.68703972 0.07341239 0.97501697 0.9240787  0.18762631
0.13694772 0.27224491 0.99779973 0.72952241 0.99078465 0.00260603
0.93639342 0.563606  ]
[0.20557396 0.14657593 0.92096736 0.56262378 0.14806236 0.55632757
0.64006382 0.56614502 0.82127183 0.03730854 0.12247717 0.22244833
0.66733879 0.30961418 0.36826784 0.23031899 0.45711542 0.58896322
0.92096479 0.97468505]]

进程已结束，退出代码为 0
```

03

第三步：训练与评估策略 —— 保障泛化性（解决“小样本过拟合”）

二、交叉验证：StratifiedKFold 十折分层抽样 + 重复实验

核心逻辑：交叉验证能保证每个折中不同类别的样本比例与原始数据集一致，避免因类别分布不均导致的评估偏差。
多次重复交叉验证过程并取平均值，减少随机因素对结果的影响，使评估更稳定可靠。

ResMTN2 - 副本.py | Activate.py | Initialization.py | Gradient_renewal.py | lunwen 22.py

```python
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression  # 假设用逻辑回归模型，实际替换为你的神经网络

# 模拟二分类标签
Labels = np.random.randint(low=0, high=2, size=100)

# 模拟脑电训练数据（res_Train_Data_normalized），形状为 [样本数, 特征数]
# 这里假设是 100 个样本，每个样本有 20 个特征，实际应替换为真实的脑电数据
res_Train_Data_normalized = np.random.randn(100, 20)

# 初始化十折分层交叉验证器
skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

testNum = 10  # 重复实验次数
Total_acc = 0.0

for repeat in range(testNum):
```

运行 lunwen 33

```
Repeat 5, Average Fold Accuracy: 0.5000
Repeat 6, Average Fold Accuracy: 0.5000
Repeat 7, Average Fold Accuracy: 0.5000
Repeat 8, Average Fold Accuracy: 0.5000
Repeat 9, Average Fold Accuracy: 0.5000
Repeat 10, Average Fold Accuracy: 0.5000
Total Average Accuracy: 0.5000

进程已结束，退出代码为 0
```

06

Experimental Results

```
(myenv) PS E:\论文\ResMTN for EEG - 副本> python "ResMTN2 - 副本.py"
E:\论文\ResMTN for EEG - 副本\ResMTN2 - 副本.py:99: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  res_Train_Data_valid = torch.tensor(res_Train_Data,
No.1, K=1, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=2, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=3, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=4, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=5, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=6, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=7, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=8, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=9, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=10, Train accurancy:100.00%, Test accurancy:100.00%
E:\论文\ResMTN for EEG - 副本\ResMTN2 - 副本.py:99: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  res_Train_Data_valid = torch.tensor(res_Train_Data,
No.2, K=1, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=2, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=3, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=4, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=5, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=6, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=7, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=8, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=9, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=10, Train accurancy:100.00%, Test accurancy:100.00%
E:\论文\ResMTN for EEG - 副本\ResMTN2 - 副本.py:99: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  res_Train_Data_valid = torch.tensor(res_Train_Data,
No.3, K=1, Train accurancy:100.00%, Test accurancy:100.00%
No.3, K=2, Train accurancy:100.00%, Test accurancy:100.00%
No.3, K=3, Train accurancy:100.00%, Test accurancy:100.00%
No.3, K=4, Train accurancy:100.00%, Test accurancy:100.00%
```

核心结果表现：

准确率：在所有实验中，训练集准确率和测试集准确率**均达到 100%**。

第 1 次（No.1）、**第 2 次**（No.2）还是第 **3 次**（No.3）**等重复**实验，以及每个实验中的 **10 折交叉**验证（K=1到K=10），**模型在**训练集和测试集上的分类都完全正确。

稳定性：多次重复实验（至少 **3 次**）**和多折交叉**验证的结果高度一致，**没有出**现准确率波动，说明模型**性能不受数据划分或随机因素影响**，稳定性极佳。

```
(myenv) PS E:\论文\ResMTN for EEG - 副本> python "ResMTN2 - 副本.py"
E:\论文\ResMTN for EEG - 副本\ResMTN2 - 副本.py:99: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  res_Train_Data_valid = torch.tensor(res_Train_Data,
No.1, K=1, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=2, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=3, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=4, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=5, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=6, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=7, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=8, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=9, Train accurancy:100.00%, Test accurancy:100.00%
No.1, K=10, Train accurancy:100.00%, Test accurancy:100.00%
E:\论文\ResMTN for EEG - 副本\ResMTN2 - 副本.py:99: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  res_Train_Data_valid = torch.tensor(res_Train_Data,
No.2, K=1, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=2, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=3, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=4, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=5, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=6, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=7, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=8, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=9, Train accurancy:100.00%, Test accurancy:100.00%
No.2, K=10, Train accurancy:100.00%, Test accurancy:100.00%
E:\论文\ResMTN for EEG - 副本\ResMTN2 - 副本.py:99: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  res_Train_Data_valid = torch.tensor(res_Train_Data,
No.3, K=1, Train accurancy:100.00%, Test accurancy:100.00%
No.3, K=2, Train accurancy:100.00%, Test accurancy:100.00%
No.3, K=3, Train accurancy:100.00%, Test accurancy:100.00%
No.3, K=4, Train accurancy:100.00%, Test accurancy:100.00%
```

实验结论:**泰勒多**项式展开特征，增强”“**定制化网**络组件” **和** “**十折交叉**验证 + **重复**实验” **策略**， 100% **的准确率**证明该方法能解决当前脑电信号二分类任务，有效捕捉数据中的非线性关系，且泛化能力优异。

THANK YOU FOR READING!

感谢您的观看

汇报人：金帅